EECS 4314 - Project Design Document
Group 2 - TutorHub

Authors: Amira Mohamed, Yazan Haddad, Mohsen Maoodhah, Dawood Al-Janaby, Arshia Hassanpour, Dmitrii Sidorin, Kamil Ginyatov

Contributors: Amira Mohamed, Yazan Haddad, Mohsen Maoodhah, Dawood Al-Janaby, Arshia Hassanpour, Dmitrii Sidorin, Kamil Ginyatov

Last updated: Feb 14, 2026
Status: Final

LASSONDE SCHOOL OF ENGINEERING | YORK U

Table of Contents

Purpose 3
Background Reading 3
Context 3
Detailed Design 3
Non Functional Requirements 4
Implementation Plan 8
Project Planning 9
Appendix 12

Purpose

The purpose of this project is to help students, mainly high school students, receive academic support in courses they find challenging. The platform allows students to enroll in courses, connect with tutors, and receive guided instruction and practice questions to support their learning and progress.

Background Reading

There are similar existing platforms to TutorHub like Chegg and Coursera, which TutorHub addresses some of the flaws the two existing services have.

Chegg:

- Copyrighted course material is often posted there for students to view answers and solutions, which is a form of academic dishonesty. TutorHub aims to help high school students learn and use their critical thinking skills with certified tutors, instead of instant gratification.
- Relating to tutors, there's no guarantee that they are verified to answer questions accurately, anybody can use AI to answer questions instead of interacting one-on-one with verified tutors.

Coursera:

- People enroll in courses just to get certifications / diplomas, mostly aimed at post-secondary students, which sometimes doesn't include one-on-one mentoring because thousands of students enroll in them
- The purpose of TutorHub is to enroll in courses and promote learning and mastery of high school subjects over just getting a certification in small groups.

Context

TutorHub can be used as a SaaS (Software as a Service) for tutor firms or educational companies to connect their tutors with high school students. This simplifies their contact system between tutors and students in one centralized place. This will meet the needs of TutorHub as a business and the user.

TutorHub can also be used for solo-tutors or smaller tutor groups with less students (ex. high school peer tutoring volunteering). It's easier to use than having to scramble through emails and contacts everywhere.

Detailed Design

- **The overall system architecture**

The system will follow Modular Monolith design implemented using Next.js as a full stack framework. This architecture ensures clear separation of concerns while remaining feasible within the project timeline.

1. **Presentation Layer -** Implementation will be using Next.js (TypeScript). Responsible for rendering user interfaces for Students, Tutors, and Admins. Includes dashboards, tutor browsing pages, enrollment interfaces, and assignment submission forms.
2. **API / Application Layer -** Implementation will be using next.js api routes or tRPC. It handles:
    - Authentication and session management
    - Role based access control
    - Course enrollment
    - Assignment posting and submission
    - Study plan management
3. **Business Logic Layer -** It contains service functions for:
    - Validating user roles and permissions
    - Managing enrollment logic
    - Handling assignment workflows
    - Tracking academic progress
    - Coordinating tutor-student interactions
4. **Data Layer -** PostgreSQL db, Prisma ORM for db access. Stores data such as:
    - Users and roles
    - Tutor profiles
    - Courses
    - Enrollment
    - Study Plans
    - Assignments
    - Submissions

Non Functional Requirements

In addition to functional requirements, TutorHub must satisfy several non-functional requirements to ensure system quality, reliability, and usability.

1. **Performance** - The system should respond to common user actions such as login, enrollment, browsing courses, and assignment submission within 2 seconds under normal usage conditions. The system should support at least 100 concurrent users without significant performance degradation.

2. **Security -** All protected routes must require authentication. Role-based access control must ensure that students, tutors, and admins can only access functionality appropriate to their role. User passwords must be securely stored

using hashing. Sensitive configuration data must be stored using environment variables.

3. **Availability** - The system should be available 24/7 except during scheduled maintenance. The hosting platforms (Vercel and Railway/Render) should provide reliable uptime and basic fault tolerance.

4. **Usability** - The interface should be intuitive and easy to navigate. Students should be able to enroll in courses and submit assignments without requiring external assistance. The layout should clearly separate dashboards for students, tutors, and admins.

5. **Maintainability** - The system will follow a modular monolith structure with MVC style separation of concerns. Code will be organized into clear modules (Authentication, Enrollment, Assignments, Messaging). Proper naming conventions and comments will be used where necessary to improve readability and future maintenance.

6. **Scalability** - The database schema should support increasing numbers of users, courses, and assignments without structural redesign. The architecture should allow optional separation into services in the future if the platform grows.

- **How users interact with the system**

  - **Students** can register, browse courses, book tutoring sessions, access and submit practice questions, and view their progress.
  - **Tutors** can manage tutoring sessions, upload practice questions, track student progress, and interact with students.
  - **Admin** can manage users and courses, approve tutors, and monitor system activity.

  All interactions are role-based, with the controller ensuring that each user only accesses functionality appropriate to their role.

- **How software components interact**

The users — including students, tutors, and admins — interact with the View to perform actions such as “Book a Session” or “Enroll in a Course.” The View sends these user actions to the Controller, which processes the request, applies business logic, and updates the Model as needed. The Model interacts with the database to retrieve or store data and returns the results to the Controller. The Controller then updates the View, which displays feedback to the user, such as “Session Booked!” or “You were successfully added to the course.”

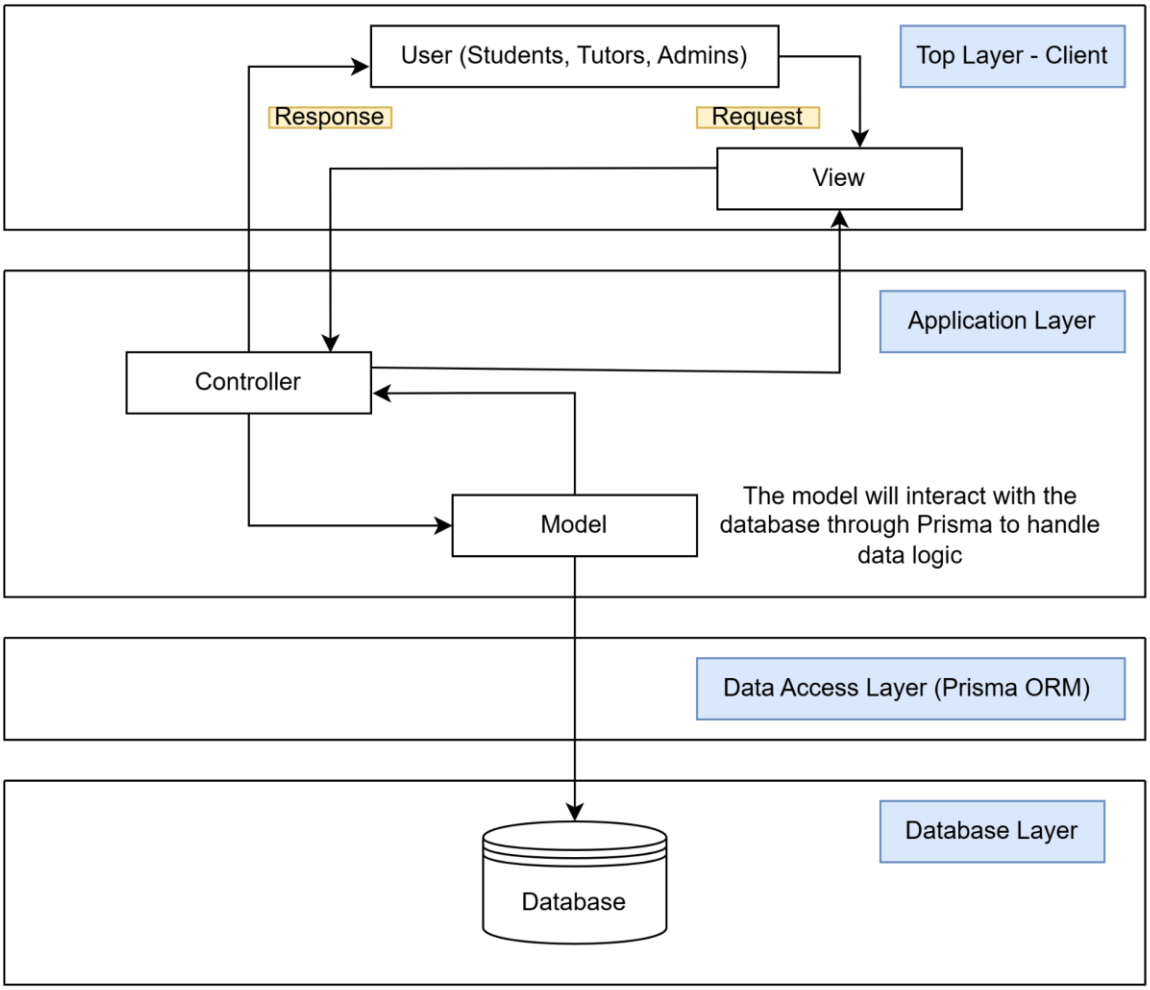

Figure 1: How the layers will interact with each other

This flow ensures a clear separation of concerns: the View handles presentation, the Controller handles application logic, and the Model manages the data.

- **How data flows through the system?**

  Flow 1: User Registration / Account Creation

  1. The user fills out the sign-up form in the UI and clicks Register.
  2. UI sends requests to API (e.g., POST /api/auth/register).
  3. API validates inputs (email format, password rules, duplicates).
  4. Service hashes password and creates a new User record via Prisma in PostgreSQL.
  5. API returns success, UI redirects user to login/dashboard.

  Flow 2: User Login

  1. The user enters credentials and clicks Login.
  2. UI sends request to authentication endpoint (NextAuth/Auth.js).
  3. API verifies credentials against stored user data (hashed password check).

4. If valid, a session/token is issued and returned to the client.
5. UI updates to show the correct dashboard based on role (Student/Tutor/Admin).

Flow 3: Student Browses Tutors/Courses

1. The student opens the tutor browsing page and selects subject filters.
2. UI sends request
3. API queries Tutor/Course data via Prisma from PostgreSQL.
4. API returns list of tutors/courses.
5. UI displays results to the student.

Flow 4: Student Enrolls in a Course

1. The student logs in and browses available tutors/courses on the Next.js UI (Presentation Layer).
2. The student clicks “Enroll” on a course page.
3. The UI sends a request to the backend (API/Application Layer), e.g., POST /api/enrollments with the course ID.
4. The API/controller checks:
    - the user is authenticated,
    - the user role is Student,
    - the student is not already enrolled in the course.
5. The business logic calls the data access layer (Prisma ORM) to create an Enrollment record in PostgreSQL.
6. The database returns success, and the API sends a success response back to the UI.
7. The UI updates to show the student is enrolled and can now access course content.

Flow 5: Student Submits an Assignment

1. The student opens a course and selects an assignment on the UI.
2. The student uploads/enters their work and clicks “Submit”.
3. The UI sends a request to the backend, e.g., POST /api/submissions with assignment ID and submission data.
4. The API/controller checks:
    - the user is authenticated,
    - the user role is Student,
    - the student is enrolled in that course,
    - the assignment exists and is open for submission.
5. The service stores the submission using Prisma ORM in PostgreSQL (creates a Submission record).
6. The API returns a confirmation response to the UI.
7. The UI shows “Submitted” and the tutor can later view/review the submission.

Implementation Plan

- **Technologies you intend to use**
  - TypeScript and Next.js as the frontend / full-stack framework
  - Authentication: NextAuth/Auth.js
  - Backend endpoints: Next.js API Routes (REST) or tRPC
  - Database: PostgreSQL
  - ORM: Prisma
  - Tools: Github, VS code, Postman, Jira for tracking

- **How you will manage source code**

  - We will use one GitHub repository for the whole project.
  - We will use 2 main branches: main and dev. Main will be used as a final working version, only stable code goes there, and dev will hold the work which is in progress.
  - When someone starts a feature, they create a new branch with their name and task they are working on. When the feature is done, they merge it into dev.
  - Nobody commits directly to main.
  - We only put code into main when it's tested and ready.

- **How you will integrate components**

  - We will build the project in this order:
    1. Database + Prisma schema (tables/models like User, Course, Enrollment, Assignment)
    2. Backend API routes (enroll, post assignment, submit assignment)
    3. Frontend pages (connect buttons/forms to the API)
  - We will agree on the API I/O so the frontend and backend match.

- **How you intend to test the system**

  - We will test the main user flows:
    1. Student logs in
    2. Student browses tutors/courses
    3. Student enrolls in a course
    4. Tutor posts an assignment
    5. Student submits as assignment
  - We will also test role rules:
    1. Students can’t do tutor/admin actions
    2. Tutor cant do admin actions
  - We will do:
    1. Quick manual testing on the website
    2. Basic API testing under postman client

- **How you will deploy the system**

  - We will deploy the web app using Vercel
  - We will host the database using Railway/render PostgreSQL
  - We will store secret keys in .env
  - After deployment, we will do a final check to make sure:
    1. Login works
    2. Enrollment works
    3. Assignment submission works

Project Planning

- **Team organization**

  - **Who does what?**

TutorHub is developed collaboratively by seven team members. Responsibilities are divided by primary system areas based on each member's strengths and experience. These assignments represent ownership and accountability, but development remains collaborative. All team members contribute across modules, support integration efforts, and participate in testing and code reviews.

- Primary area ownership is structured as follows:
- Frontend development (Arshia Hassanpour): UI components, dashboards, role-based views
- Backend API development (Yazan Haddad): business logic and API endpoints
- Database design and Prisma schema (Mohsen Maoodhah): schema modeling and data integrity
- Authentication and authorization (Kamil Ginyatov): role-based access and session management
- Deployment and environment configuration (Dawood Al-Janaby): Vercel setup and production environment
- Recommendation service development (Amira Mohamed): Python FastAPI service
- Project coordination and milestone tracking (Dawood Al-Janaby): sprint tracking and documentation alignment

Although each member has a primary ownership area, the team follows a collaborative development approach. Major features are reviewed collectively, integration testing is performed jointly, and members assist one another when cross module dependencies arise.

  - **How work is coordinated?**

The team follows a lightweight Agile approach.

- Work is organized into Epics aligned with major system features such as Authentication, Course Management, Enrollment, Assignment Workflow, and Messaging.
- Each Epic is divided into smaller User Stories representing functional requirements.
- Tasks are tracked using Jira.
- Development is done in feature branches and merged into the development branch after peer review.
- Weekly meetings are held to review progress and adjust workload if necessary.

This approach ensures structured progress while remaining adaptable to project constraints.

- **Schedule**
    - Timing for major development tasks
    - Development milestones

Phase1: Architecture & core Setup

Timeline: Before design document submission (Week of Feb7)

Goals:

- Finalize system architecture (Modular Monolith + MVC separation)
- Define database schema (Users, Courses, Enrollment, Assignments)
- Set up GitHub repository and branching strategy
- Configure development environment
- Prepare initial UI layout structure

Deliverable: Completed and submitted Design Document

Phase 2: Core System Features

Timeline: Feb 8 – Feb 23 (Before Check-In 1)

Goals:

- Implement authentication and role-based access control
- Implement course browsing and enrollment logic
- Create basic dashboards for students and tutors
- Connect frontend to backend API routes
- Initial integration testing of main flows

Milestone: Demonstrate working login and enrollment workflow at Check-In 1

Phase 3: Assignment Workflow & Advanced Features

Timeline: Feb 24 – March 16 (Before Check-In 2 and 3)

Goals:

- Implement assignment posting and submission workflow
- Implement study plan management
- Implement messaging functionality
- Integrate optional Python recommendation service
- Perform module integration testing

Milestones: Check-In 2: Demonstrate assignment creation and submission Check-In 3: Demonstrate integrated system with multiple roles

Phase 4: Stabilization, Testing & Deployment

Timeline: March 17 – March 30

Goals:

- Full system integration testing
- Fix identified bugs
- Improve UI consistency
- Deploy system to Vercel
- Verify production database functionality
- Prepare presentation demo

Milestone: Stable deployed system ready for Project Presentation

Final Deliverables

- Project Presentation - Week of March 31
- Final Report Submission - Week of March 31

Risk Management

The team has identified risks are primarily integrating front-end, back-end, and database components, scheduling delays due to academics, added feature creep (scope creep), deployment failures in the production environment, etc. The team will mitigate these risks by establishing API contracts in advance, doing weekly integration tests of all components, prioritizing core system requirements before optional enhancements, redistributing tasks if we encounter delays during implementation (milestone based), and performing pre-deployment testing on Vercel to assure environment stability before the final presentation occurs.

Appendix

The TutorHub project repository is available at:
https://github.com/kamilsft/TutorHub/tree/main